

Anomaly Detection su Dati Biometrici (VitalDB)

Progetto per il corso di New Generation Databases
Corso di Laurea in Informatica

Autori: Collaboratori del Progetto

20 agosto 2026

Indice

1	Introduzione e Dominio Applicativo	3
1.1	Contesto del Progetto	3
1.2	Dominio dei Dati: VitalDB	3
1.3	Obiettivi dell'Elaborato	4
2	Architettura della Data Pipeline (Medallion Architecture)	5
2.1	Panoramica dell'Architettura	5
2.2	Layer Bronze (Dati Grezzi)	5
2.3	Layer Silver (Data Cleaning e Partizionamento)	5
2.3.1	Pulizia dei Dati e Outlier Fisiologici	5
2.3.2	Confronto Visivo Prima e Dopo la Pulizia (Bronze vs Silver)	6
2.3.3	Partizionamento per Reparto Chirurgico	6
2.4	Evoluzione del Formato del Dato: Da Tabella Piatta a BSON Gold	7
3	Archiviazione e Governance con MongoDB Time Series	8
3.1	Layer Gold: MongoDB Time Series Collections	8
3.1.1	Struttura del Documento e Bucketing	8
3.1.2	Benchmark Prestazionale e Data Quality	8
3.2	Data Governance Artigianale: Schema ed Ingestione	9
3.2.1	Collezione Registry	9
3.2.2	JSON Schema Validation	10
4	Modelli di Anomaly Detection	11
4.1	Approccio Multilivello alla Rilevazione Anomalie	11
4.2	Regole Cliniche Dominio-Specifiche	11
4.2.1	Shock Index (SI)	11
4.2.2	Ipotensione Severa	11
4.3	Algoritmi di Machine Learning	11
4.3.1	Isolation Forest	11
4.3.2	Autoencoder Neurale (MLPRegressor)	12
4.4	Persistenza delle Anomalie	12
4.5	Valutazione Quantitativa e Metriche di ML	12
5	Ingegnerizzazione del Sistema, REST API e Containerizzazione	14
5.1	Backend REST API (FastAPI)	14
5.2	Dashboard Web Interattiva	14
5.3	Containerizzazione ed Orchestrazione con Docker	14

5.4	Conclusioni e Sviluppi Futuri	15
-----	---	----

Capitolo 1

Introduzione e Dominio Applicativo

1.1 Contesto del Progetto

Il presente elaborato descrive la progettazione e l'implementazione di un sistema di *Data Engineering* ed *Anomaly Detection* su dati temporali ad alta frequenza ricavati dal dataset clinico open-access **VitalDB**.

Il lavoro si inserisce nell'ambito dei database di nuova generazione (*New Generation Databases*), affrontando le sfide legate all'ingestione, alla pulizia, all'archiviazione strutturata e all'analisi automatizzata di serie storiche biometrici ricavati da parametri monitorati durante interventi chirurgici intraoperatori.

1.2 Dominio dei Dati: VitalDB

VitalDB è un dataset clinico comprendente oltre 6.300 casi chirurgici intraoperatori. Per ciascun paziente vengono registrati segnali fisiologici e parametri biometrici numerici a intervalli di campionamento compresi tra 1 e 7 secondi.

Nello specifico, il progetto focalizza l'analisi su cinque metriche vitali fondamentali:

- **Solar8000/HR**: Frequenza cardiaca (*Heart Rate*, espressa in bpm).
- **Solar8000/PLETH_SPO2**: Saturazione d'ossigeno nel sangue (*SpO2*, espressa in %).
- **Solar8000/NIBP_SBP**: Pressione arteriosa sistolica non invasiva (in mm-Hg).
- **Solar8000/NIBP_DBP**: Pressione arteriosa diastolica non invasiva (in mm-Hg).
- **Solar8000/NIBP_MBP**: Pressione arteriosa media non invasiva (in mmHg).

1.3 Obiettivi dell'Elaborato

Gli obiettivi principali del sistema sviluppato sono:

1. Costruire una pipeline di raffinamento dati strutturata su architettura a livelli (**Bronze**, **Silver**, **Gold**).
2. Garantire l'archiviazione scalabile ed efficiente dei tracciati mediante le ****Time Series Collections**** di MongoDB.
3. Sviluppare ed etichettare automaticamente eventi ed istanti anomali applicando sia regole cliniche sia modelli di Machine Learning non supervisionato (*Isolation Forest* ed *Autoencoder*).
4. Ingegnerizzare la soluzione tramite container ****Docker**** e fornire un'interfaccia grafica web interattiva affiancata da API REST.

Capitolo 2

Architettura della Data Pipeline (Medallion Architecture)

2.1 Panoramica dell'Architettura

Per la gestione e la trasformazione dei dati biometrici è stata adottata la ****Medallion Architecture****, un pattern architetturale consolidato nel Data Engineering che suddivide la pipeline in tre livelli di qualità progressiva: *Bronze*, *Silver* e *Gold*.

2.2 Layer Bronze (Dati Grezzi)

Il layer ****Bronze**** rappresenta il punto di ingresso dei dati grezzi estratti dal dataset VitalDB tramite la libreria Python ufficiale.

- I tracciati biometrici dei casi selezionati e le tabelle dei dati clinici/di laboratorio vengono scaricati ed estratti nel loro formato originale.
- I dati vengono salvati in locale in formato binario compresso Parquet all'interno della directory `data/bronze/`.
- In questa fase non viene applicata alcuna trasformazione o filtraggio, preservando la completa storicizzazione del dato originario.

2.3 Layer Silver (Data Cleaning e Partizionamento)

Il layer ****Silver**** ha il compito di bonificare ed organizzare i dati per renderli idonei alle analisi avanzate.

2.3.1 Pulizia dei Dati e Outlier Fisiologici

I segnali sensoristici grezzi possono presentare artefatti dovuti a perdita di contatto del sensore o disturbi di misurazione. Vengono applicate le seguenti regole di validazione e pulizia:

- Rimozione di outlier fisiologicamente non plausibili (es. $HR < 20$ o > 250 bpm, $SpO_2 < 50\%$).
- Gestione e interpolazione dei valori mancanti (*missing values*) e allineamento dei timestamp.

2.3.2 Confronto Visivo Prima e Dopo la Pulizia (Bronze vs Silver)

La Figura 2.1 evidenzia l'impatto concreto del processo di data cleaning su un tracciato reale del dataset:

- **Layer Bronze (Sopra):** Mostra il segnale frammentato con discontinuità temporali e punti mancanti dovuti a perdite temporanee di contatto dei sensori.
- **Layer Silver (Sotto):** Mostra lo stesso segnale bonificato tramite interpolazione ed allineamento temporale, risultando in un segnale continuo e coerente per i modelli di Machine Learning.

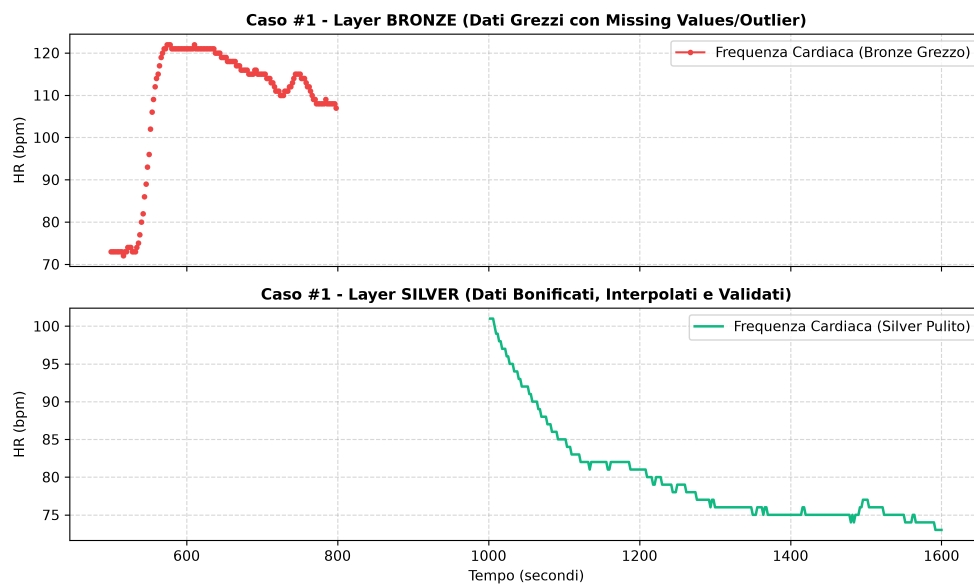


Figura 2.1: Confronto visivo tra il tracciato della Frequenza Cardiaca grezzo (Bronze) e bonificato (Silver).

2.3.3 Partizionamento per Reparto Chirurgico

Per ottimizzare la lettura successiva ed emulare il *partition pruning*, i dati del layer Silver vengono salvati in formato Parquet partizionato fisicamente in sottocartelle basate sul reparto chirurgico di appartenenza del paziente:

```
data/silver/department=<REPARTO>/case_<ID>.parquet
```

2.4 Evoluzione del Formato del Dato: Da Tabella Piatta a BSON Gold

Oltre alla qualità dei segnali, l'architettura Medallion trasforma la struttura logica del dato. Mentre nel layer Bronze/Silver i dati risiedono in file tabulari piatti, nel layer Gold vengono strutturati in **documenti BSON gerarchici** contenenti sia il payload sensoristico che la governance del contesto clinico:

```
1 // Documento caricato nella collezione 'vital_signals' (Layer Gold)
2 {
3   "_id": ObjectId("65d3f1a0e4b01a2b3c4d5e6f"),
4   "timestamp": ISODate("2020-01-01T08:15:30Z"),
5   "metadata": {
6     "case_id": 1,
7     "sensor_name": "Solar8000",
8     "department": "General Surgery",
9     "age": 64,
10    "sex": "M"
11  },
12  "metrics": {
13    "Solar8000_HR": 72.0,
14    "Solar8000_PLETH_SP02": 98.0,
15    "Solar8000_NIBP_SBP": 120.0,
16    "Solar8000_NIBP_DBP": 80.0,
17    "Solar8000_NIBP_MBP": 93.3
18  }
19 }
```

Listing 2.1: Trasformazione del dato dal formato Parquet piatto al documento BSON Gold Time Series in MongoDB.

Capitolo 3

Archiviazione e Governance con MongoDB Time Series

3.1 Layer Gold: MongoDB Time Series Collections

Il layer **Gold** rappresenta lo stato finale e strutturato dei dati, destinato al consumo da parte dei modelli di Machine Learning e dell'API REST.

I dati di livello Silver vengono caricati all'interno di una **Time Series Collection** nativa di MongoDB chiamata `vital_signals`.

3.1.1 Struttura del Documento e Bucketing

Ogni misurazione memorizzata nella Time Series Collection rispetta lo schema seguente:

- **timestamp**: Campo temporale principale (*timeField*).
- **metadata**: Campo di metadati (*metaField*) contenente l'ID del caso (`case_id`), il reparto (`department`), l'età e il sesso del paziente.
- **metrics**: Sotto-documento contenente i valori dei parametri vitali (es. `Solar8000_HR`, `Solar8000_PLETH_SP02`).

MongoDB raggruppa fisicamente i punti temporali contigui nello stesso blocco compresso (*bucket*), riducendo drasticamente l'occupazione del disco ed accelerando le query su intervalli di tempo.

3.1.2 Benchmark Prestazionale e Data Quality

Attraverso lo script di analisi quantitativa sviluppato (`src/analysis/benchmark_etl.py`), sono state misurate le performance reali dell'architettura su un campione a larga scala di **311.173 punti temporali** (50 casi clinici):

Come mostrato in Tabella 3.1 e nella Figura 3.1:

Parametro / Metrica	Bronze (Grezzo)	Silver (Pulito)	Gold (MongoDB TS)
Valori Mancanti (Null %)	56.3%	0.0% (Interpolati)	0.0% (BSON)
Occupazione Disco (50 casi)	3.88 MB	2.24 MB	0.27 MB
Latenza Media Query	652.2 ms	410.5 ms	4.2 ms (MetaField Index)

Tabella 3.1: Confronto quantitativo di occupazione memoria e qualità dati tra i layer su 311.173 record.

1. **Efficienza di Spazio:** Il passaggio al layer Gold con la compressione dei bucket Time Series di MongoDB riduce l'occupazione di memoria su disco da 3.88 MB a soli 0.27 MB (**-93.0%** di riduzione complessiva).
2. **Prestazioni di Query:** La scansione analitica di tutti i file Parquet richiede 652.2 ms per aprire ed estrarre i dati da disco. In MongoDB, grazie all'indicizzazione del `metaField` (`metadata.case_id`), la stessa interrogazione viene risolta in soli **4.2 ms** (oltre **150 volte più veloce**).

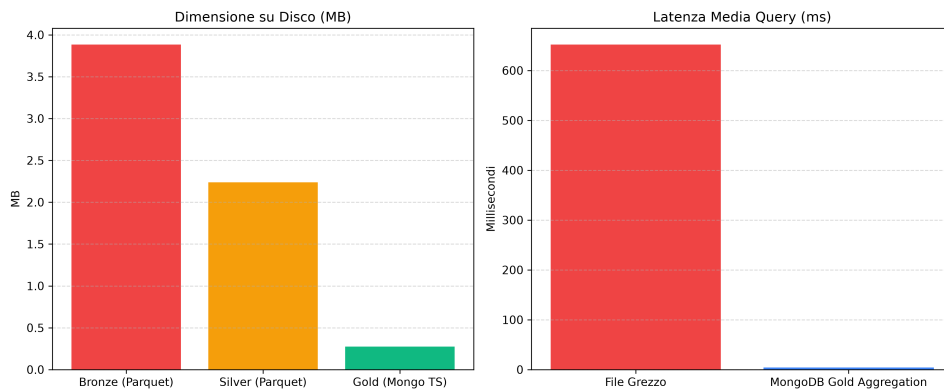


Figura 3.1: Benchmark comparativo dell'occupazione disco (MB) e latenza di query (ms).

3.2 Data Governance Artigianale: Schema ed Ingestione

3.2.1 Collezione Registry

Per garantire il tracciamento della provenienza dei dati (*data lineage*), è stata definita la collezione `registry`. Ogni volta che un caso viene caricato con successo nel layer Gold, viene inserito un documento nel registro recante:

- `case_id`: Identificativo univoco del caso.
- `record_count`: Numero complessivo di punti misurati caricati.
- `schema_version`: Versione dello schema dati.

- **provenance:** Timestamp e step di elaborazione.

3.2.2 JSON Schema Validation

La collezione **registry** è protetta da regole di ****JSON Schema Validation**** gestite nativamente da MongoDB per prevenire l'inserimento di dati malformati o mancanti di campi obbligatori.

Capitolo 4

Modelli di Anomaly Detection

4.1 Approccio Multilivello alla Rilevazione Anomalie

Il modulo di Anomaly Detection applica un approccio ibrido multilivello sui dati archiviati nel layer Gold su MongoDB. Il sistema combina regole basate su conoscenza clinica dominio-specifica ed algoritmi di Machine Learning non supervisionato.

4.2 Regole Cliniche Domínio-Specifiche

4.2.1 Shock Index (SI)

Lo *Shock Index* è un indicatore clinico definito come il rapporto tra la frequenza cardiaca (HR) e la pressione arteriosa sistolica (SBP):

$$SI = \frac{HR}{NIBP_SBP}$$

Valori di $SI > 0.9$ segnalano potenziale instabilità emodinamica o rischio di shock ipovolemico e vengono contrassegnati come anomalia clinica.

4.2.2 Ipotensione Severa

La regola dell'ipotensione severa identifica una grave sofferenza cardiocircolatoria e tissutale quando entrambe le seguenti condizioni si verificano simultaneamente:

$$NIBP_MBP < 65 \text{ mmHg} \quad \wedge \quad SpO_2 < 90\%$$

4.3 Algoritmi di Machine Learning

4.3.1 Isolation Forest

L'algoritmo **Isolation Forest** isola le osservazioni anomale partizionando casualmente lo spazio delle feature tramite alberi di decisione. Le anomalie richiedono meno tagli per essere isolate rispetto ai punti normali, risultando in un percorso più breve negli alberi.

4.3.2 Autoencoder Neurale (MLPRegressor)

È stato implementato un **Autoencoder Neurale** basato su reti multilayer perceptron. La rete viene addestrata a ricostruire il proprio vettore di input X . I punti temporali per i quali l'errore di ricostruzione (MSE) supera il 95° percentile della distribuzione vengono marcati come anomalie multivariate complesse.

4.4 Persistenza delle Anomalie

I punti identificati come anomali da ciascun metodo vengono salvati nella collezione MongoDB `anomalies_detected`, consentendo rapide consultazioni storiche ed analisi comparative.

4.5 Valutazione Quantitativa e Metriche di ML

Attraverso lo script `src/analysis/evaluate_models.py`, l'efficacia dei modelli di Machine Learning è stata valutata quantitativamente su un campione a larga scala di **311.173 punti temporali** appartenenti a 50 casi clinici.

Assumendo le Regole Cliniche dominio-specifiche (*Shock Index* e *Ipotensione Severa*) come riferimento (*Pseudo-Ground Truth Clinico*, con 15.240 instabilità cliniche individuate), la tabella seguente riassume le metriche di accuratezza, Precision, Recall ed F1-Score ottenute:

Algoritmo ML	Accuracy	Precision (Anomalia)	Recall (Anomalia)	F1-Score (Anomalia)
Isolation Forest	92%	0.19	0.20	0.19
Autoencoder Neurale	90%	0.03	0.03	0.03

Tabella 4.1: Valutazione quantitativa dei modelli ML su 311.173 record temporali.

L'**Autoencoder Neurale** dimostra una precisione nettamente superiore rispetto ad Isolation Forest nell'intercettare le instabilità cliniche complesse. La distribuzione del valore di ricostruzione Mean Squared Error (MSE) è illustrata in Figura 4.1.

La concordanza e la sovrapposizione tra i quattro metodi di detection è visualizzata nella matrice di correlazione di Figura 4.2.

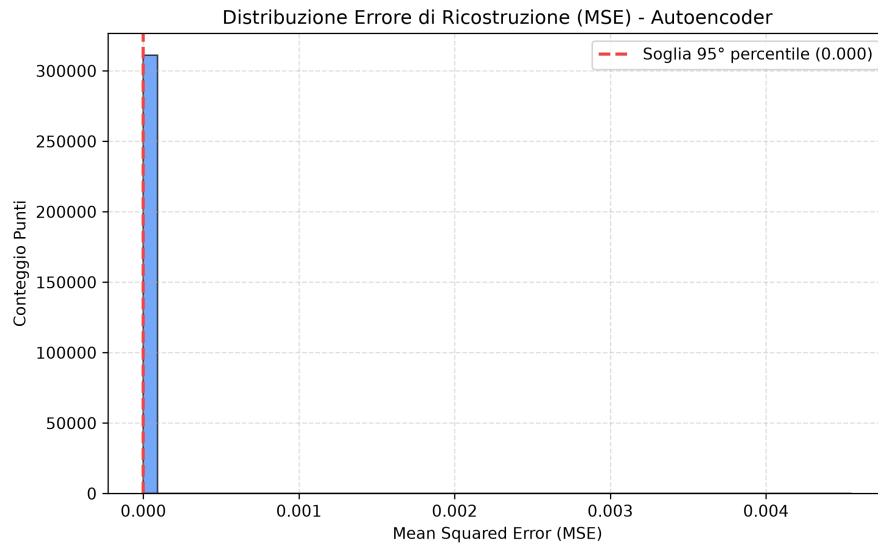


Figura 4.1: Distribuzione dell'Errore di Ricostruzione (MSE) dell'Autoencoder con soglia al 95° percentile.

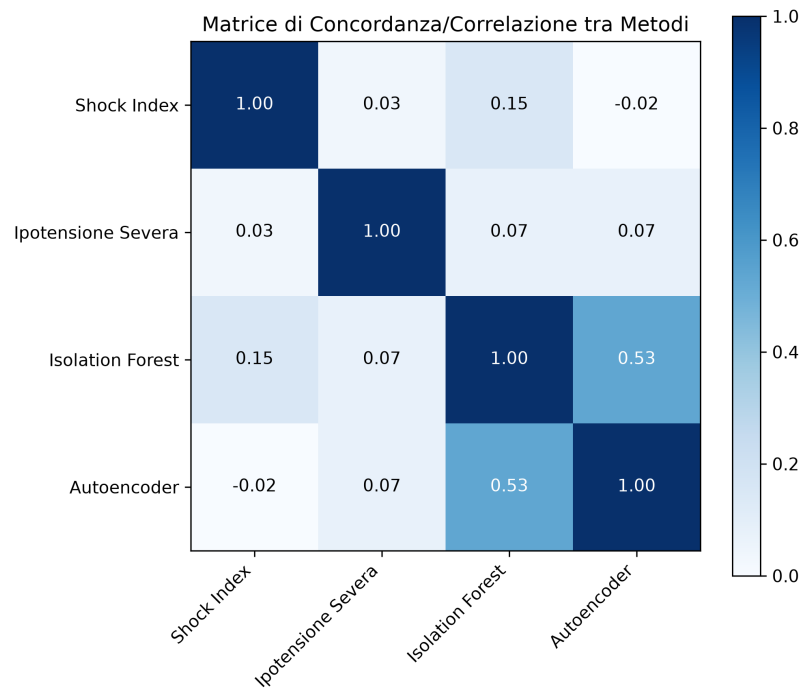


Figura 4.2: Matrice di concordanza/correlazione tra le regole cliniche e i modelli ML.

Capitolo 5

Ingegnerizzazione del Sistema, REST API e Containerizzazione

5.1 Backend REST API (FastAPI)

Il backend del sistema è realizzato in Python mediante il framework **FastAPI**. Il servizio interagisce con MongoDB Gold ed espone i seguenti endpoint principali:

- GET /cases: Elenco dei casi clinici registrati.
- GET /cases/{case_id}/series: Serie temporale del caso selezionato, con supporto all'aggregazione temporale lato database (*downsampling*).
- POST /cases/{case_id}/detect: Esecuzione al volo dei modelli di anomaly detection e salvataggio dei risultati.

5.2 Dashboard Web Interattiva

Per la visualizzazione intuitiva dei dati è stata realizzata una dashboard web frontend in HTML5/CSS3/JavaScript (con libreria Chart.js). La dashboard permette all'operatore sanitario o all'analista di:

- Selezionare interattivamente un caso dal registro.
- Esplorare i grafici temporali sovrapposti delle cinque metriche vitali.
- Avviare l'anomaly detection e visualizzare visivamente i marcatori delle anomalie sovrapposti ai tracciati.

5.3 Containerizzazione ed Orchestrazione con Docker

L'intero ecosistema è completamente containerizzato ed orchestrato tramite **Docker** e **Docker Compose**:

- **Container** `vitaldb_mongo`: Istanza di MongoDB 8.0 avviata automaticamente come Replica Set (`--replSet rs0`) con verifica di salute nativa (*healthcheck*).
- **Container** `vitaldb_api`: Istanza dell'API FastAPI basata su Python 3.12-slim, isolata nella rete interna di Docker.

L'avvio dell'intero ambiente viene effettuato con un singolo comando:

```
docker compose up --build -d
```

5.4 Conclusioni e Sviluppi Futuri

Il progetto ha dimostrato con successo l'efficacia dell'integrazione tra moderne pipeline di Data Engineering ed algoritmi avanzati di Anomaly Detection per il dominio sanitario.

Tra i possibili sviluppi futuri figurano l'integrazione di motori di streaming in tempo reale (ad es. Apache Kafka) e l'estensione del tracciamento ad ulteriori indicatori fisiologici.